Manthan Ankolekar
Posted on 6 juin 2024 • Edited on 4 févr.

44

Building a CRUD Application with Node.js, Express, and MySQL

#webdev #javascript #beginners #programming

CRUD Applications (5 Part Series)

1. Building a CRUD Application with Node.js, Express, and MongoDB
2. **Building a CRUD Application with Node.js, Express, and MySQL**
3. Building a CRUD Application with NestJS and MongoDB
4. Building a CRUD Application with Flask and SQLAlchemy
5. Building a CRUD Application with Flask and MongoDB

In this blog, we'll walk through creating a simple CRUD (Create, Read, Update, Delete) application using Node.js, Express, and MySQL. This tutorial will guide you through setting up the project, configuring the database, and implementing the CRUD operations.

Project Setup

Step 1: Initializing the Project

Create a new directory for your project and initialize it with npm:

```
mkdir crud-nodejs
cd crud-nodejs
npm init -y
```

Install the necessary dependencies:

```
npm install express mysql2 dotenv body-parser
npm install --save-dev nodemon
```

Step 2: Project Structure

Create the following project structure:

```
crud-nodejs
├── config
│   └── database.js
├── controllers
│   └── todoController.js
├── middleware
│   └── errorMiddleware.js
├── models
│   └── todo.js
├── routes
│   └── todoRoutes.js
├── .env.example
├── index.js
└── package.json
```

Step 3: Configuring Environment Variables

Create a `.env` file (copy from `.env

.example`):

```
cp .env.example .env
```

Fill in your MySQL database credentials in the `.env` file:

```
PORT=3000
DB_HOST=localhost
DB_USER=your_user
DB_PASSWORD=your_password
DB_DATABASE=your_database
```

Step 4: Connecting to MySQL

In `config/database.js`, we set up the MySQL connection using the `mysql2` package:

```
const mysql = require('mysql2');
```

```
require('dotenv').config();

const connection = mysql.createConnection({
    host: process.env.DB_HOST,
    user: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_DATABASE
});

connection.connect((err) => {
    if (err) throw err;
    console.log('Connected to MySQL database');
});

module.exports = connection;
```

Step 5: Creating the Express Server

In `index.js`, we configure the Express server and set up the routes and error handling middleware:

```
const express = require('express');
const bodyParser = require('body-parser');
const todoRoutes = require('./routes/todoRoutes');
const errorMiddleware = require('./middleware/errorMiddleware');

require('dotenv').config();

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware
app.use(bodyParser.json());

// Routes
app.use('/todos', todoRoutes);

// Error middleware
app.use(errorMiddleware);

// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Step 6: Defining the Todo Model

In `models/todo.js`, we define the functions to interact with the MySQL database:

```
const db = require('../config/database');

exports.getAllTodos = function(callback) {
    db.query('SELECT * FROM todos', callback);
};

exports.getTodoById = function(id, callback) {
    db.query('SELECT * FROM todos WHERE id = ?', [id], callback);
};

exports.createTodo = function(newTodo, callback) {
    db.query('INSERT INTO todos SET ?', newTodo, callback);
};

exports.updateTodo = function(id, updatedTodo, callback) {
    db.query('UPDATE todos SET ? WHERE id = ?', [updatedTodo, id], callback);
};

exports.deleteTodo = function(id, callback) {
    db.query('DELETE FROM todos WHERE id = ?', [id], callback);
};
```

Step 7: Creating the Controller

In `controllers/todoController.js`, we define the logic for handling CRUD operations:

```
const Todo = require('../models/todo');

exports.getAllTodos = function(req, res) {
    Todo.getAllTodos((err, todos) => {
        if (err) throw err;
        res.json(todos);
    });
};

exports.getTodoById = function(req, res) {
    Todo.getTodoById(req.params.id, (err, todo) => {
        if (err) throw err;
        res.json(todo);
    });
};

exports.createTodo = function(req, res) {
    const newTodo = {
        title: req.body.title,
```

```
        completed: req.body.completed
    };

    Todo.createTodo(newTodo, (err, result) => {
        if (err) throw err;
        res.json({ message: 'Todo created successfully' });
    });
};

exports.updateTodo = function(req, res) {
    const updatedTodo = {
        title: req.body.title,
        completed: req.body.completed
    };

    Todo.updateTodo(req.params.id, updatedTodo, (err, result) => {
        if (err) throw err;
        res.json({ message: 'Todo updated successfully' });
    });
};

exports.deleteTodo = function(req, res) {
    Todo.deleteTodo(req.params.id, (err, result) => {
        if (err) throw err;
        res.json({ message: 'Todo deleted successfully' });
    });
};
```

Step 8: Defining Routes

In `routes/todoRoutes.js`, we set up the routes for the Todo API:

```
const express = require('express');
const router = express.Router();
const todoController = require('../controllers/todoController');

// Routes
router.get('/', todoController.getAllTodos);
router.get('/:id', todoController.getTodoById);
router.post('/', todoController.createTodo);
router.put('/:id', todoController.updateTodo);
router.delete('/:id', todoController.deleteTodo);

module.exports = router;
```

Step 9: Error Handling Middleware

In `middleware/errorMiddleware.js`, we define a simple error handling middleware:

```
module.exports = function errorHandler(err, req, res, next) {
    console.error(err.stack);
    res.status(500).send('Something broke!');
};
```

Step 10: Running the Application

Add the following scripts to `package.json`:

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "dev": "nodemon index.js",
  "start": "node index.js"
}
```

Start the application in development mode:

```
npm run dev
```

Conclusion

You now have a fully functional CRUD application built with Node.js, Express, and MySQL. This application allows you to create, read, update, and delete Todo items. This basic structure can be expanded and customized to fit more complex requirements. Happy coding!

Exploring the Code

Visit the [GitHub repository] to explore the code in detail.

CRUD Applications (5 Part Series)

1. Building a CRUD Application with Node.js, Express, and MongoDB
2. **Building a CRUD Application with Node.js, Express, and MySQL**
3. Building a CRUD Application with NestJS and MongoDB

4 Building a CRUD Application with Flask and SQLAlchemy

5 Building a CRUD Application with Flask and MongoDB

Kinde PROMOTED

Make ship happen

Kinde

Read More

Top comments (0)

Code of Conduct • Report abuse

Kinde PROMOTED

Make ship happen

npm i @kinde-oss/kinde-typescript-sdk

Kinde

Build the product, not the plumbing—JavaScript first

Kinde handles the boring but critical stuff: auth, permissions, and billing—all from one Javascript SDK.

Get a free account

Manthan Ankolekar

Software Developer | JavaScript | Angular | Nodejs | MongoDB | Express.js | Python | Firebase | MySQL | Postgresql |

LOCATION
Karnataka, India

EDUCATION
Bachelor of Engineering in Computer Science

WORK
Software Developer

JOINED
21 févr. 2021

More from Manthan Ankolekar

🚀 Building an AI-Powered Resume Optimizer with Angular, Node.js & Gemini
#webdev #programming #beginners #angular

Exploring Binary Assignment Operators with Angular v20.1.0 🚀
#webdev #programming #angular #beginners

🚀 Build a Prompt Generator App with Angular, Tailwind CSS & Google Gemini API
#webdev #programming #angular #beginners

_SurveyJS PROMOTED •••

```
import React from "react";
import { SurveyCreator, SurveyCreatorComponent } from "survey-creator-react";
import "survey-core/survey.i18n.js";
import "survey-creator-core/survey-creator-core.i18n.js";
import "survey-core/defaultV2.css";
import "survey-creator-core/survey-creator-core.css";
import "./index.css";

const surveyJSON = {
  elements: [
    //...
    {
      type: "text",
      name: "question1",
      title: "My first question",
    },
    //...
  ],
};

function SurveyCreatorRenderComponent() {
  const options = {
    showLogicTab: true,
  };
  const creator = new SurveyCreator(options);
  creator.JSON = surveyJSON;
  return <SurveyCreatorComponent creator={creator} />;
}

export default SurveyCreatorRenderComponent;
```

```
PROBLEMS   OUTPUT   TERMINAL   PORTS

➜ workspace git:(master) ✗ npm i survey-creator-react
```

JavaScript Form Builder UI Component

Generate dynamic JSON-driven forms directly in your JavaScript app (Angular, React, Vue.js, jQuery) with a fully customizable drag-and-drop form builder. Easily integrate with any backend system and retain full ownership over your data, with no user or form submission limits.

Learn more